NEURALSPARK: DESIGN AND IMPLEMENTATION OF AN AI-POWERED E-LEARNING ECOSYSTEM WITH DUAL-MODE AUTOMATED GRADING AND OFFLINE CLASSROOM INTEGRATION

A Project Report Submitted by

Arjun Oundhkar 2303031050054

Asmit Bishnoi 2303031050063

Aniket 2303031050040

Vedant Shah 2403031057118

In Partial Fulfilment for the Award of the Degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

(Specialization in Artificial Intelligence and Data Science)

Under the Guidance of

Prof. (Dr.) XYZ

Assistant Professor, Department of AIDS

PARUL INSTITUTE OF ENGINEERING AND TECHNOLOGY

PARUL UNIVERSITY, VADODARA

Academic Year: 2025–2026

PARUL UNIVERSITY

PARUL INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Artificial Intelligence and Data Science

Certificate

This is to certify that the Project-2 (Subject Code: **203105400**) of the 7th/8th Semester, entitled **"NEURALSPARK: AI-POWERED E-LEARNING PLATFORM WITH INTELLIGENT DUAL-MODE AUTOMATED GRADING SYSTEM"** of Group No. **PIET-AIDS-08** has been successfully completed by:

ARJUN OUNDHKAR	**Enrolment No.: 2303031050054**
ASMIT BISHNOI	**Enrolment No.: 2303031050063**
ANIKET	**Enrolment No.: 2303031050040**
VEDANT SHAH	**Enrolment No.: 2403031057118**

under my guidance, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech)** in **Computer Science and Engineering (Spl. in Artificial Intelligence and Data Science)** at **Parul University** during the Academic Year **2025–2026**.

Date of Submission: _____

Prof. (Dr.) XYZ
Project Guide
Dept. of AI & Data Science

Dr. Sanjay Agal
Head of Department
Dept. of AI & Data Science

Dr. Mohammad Arif
Project Coordinator
Dept. of AI & Data Science

External Examiner
Parul University
Vadodara

Acknowledgements

"The single greatest cause of happiness is gratitude."

— Auliq Ice

We express our sincere appreciation and deepest gratitude to the many mentors, faculty members, and individuals who provided invaluable guidance, encouragement, and technical assistance throughout the conceptualization, development, and realization of the **NeuralSpark** platform.

Firstly, we thank our parents and families, whose unconditional support, understanding, and belief in our aspirations have been the bedrock of our academic pursuits.

Secondly, we express our profound gratitude to our project guide, **Prof. (Dr.) XYZ**, whose technical vision, constructive feedback, and continuous mentorship provided crucial direction during every sprint of software engineering, machine learning modeling, and system integration.

We extend our sincere thanks to **Dr. Sanjay Agal**, Head of the Department of Artificial Intelligence and Data Science, and **Dr. Mohammad Arif**, Project Coordinator, for providing computational infrastructure, laboratories, and an environment that fostered innovation.

We also thank the faculty members of the Department of Artificial Intelligence and Data Science at Parul Institute of Engineering and Technology (PIET) for imparting rigorous theoretical grounding and hands-on skills in software engineering, natural language processing, and cloud architectures.

Finally, we acknowledge all open-source communities, peer students, and contributors whose tools, libraries, and feedback elevated NeuralSpark into a complete, robust, and impactful e-learning solution.

Arjun Oundhkar **Asmit Bishnoi**
Aniket **Vedant Shah**

Department of Artificial Intelligence and Data Science
Parul Institute of Engineering and Technology, Parul University, Vadodara
April 2026

Abstract

The rapid proliferation of digital learning and computer-based competitive examinations (such as GATE Data Science & Artificial Intelligence) has highlighted significant operational bottlenecks in contemporary e-learning infrastructures. Traditional learning management systems (LMS) suffer from three critical deficiencies: (1) an inability to perform automated, qualitative, and instant assessment of subjective, open-ended assignments and handwritten student submissions; (2) excessive fragmentation across video players, discussion boards, practice repositories, and grading utilities; and (3) total vulnerability to intermittent network connectivity, which systematically disenfranchises students in rural, underserved, or bandwidth-constrained educational environments.

To resolve this assessment scalability crisis and connectivity divide, this project introduces **NeuralSpark**, an end-to-end, full-stack AI-driven e-learning ecosystem engineered specifically for modern technical education and competitive examination preparation. NeuralSpark unifies five mission-critical pillars into a single high-performance platform:

1. **Structured 8-Subject Curriculum Core:** Comprehensive syllabus alignment across Linear Algebra, Calculus, Probability & Statistics, Python Programming, Data Structures & Algorithms, Machine Learning, Artificial Intelligence, and General Aptitude.

2. **Dual-Engine Multimodal Automated Grading System:** An intelligent assessment pipeline comprising: (a) a cloud-based multimodal Vision-LLM engine capable of ingesting PDF/image handwritten submissions and evaluating them against rigorous rubric criteria; and (b) a lightweight, locally fine-tuned Transformer sequence classifier delivering zero-latency, private scoring without cloud egress.

3. **Interactive Practice & PYQ Portal:** Index-ordered past-year question bank with 4-option multiple-choice questions (MCQs), instant client-side grading, LaTeX mathematical formula rendering, and step-by-step verified explanations.

4. **Community-Driven Peer Learning Forum:** Gamified question-and-answer discussion boards with reputation scoring, solution upvoting, and verified teacher answers.

5. **Edge-First Offline Classroom Architecture:** Fully containerized local launch suite with automated database migrations, superuser bootstrapping, offline speech recognition (Vosk STT), and voice response synthesis (Piper TTS).

The backend is built upon **Django 6.0.2** and **Django Ninja**, achieving sub-50ms REST API response latencies with strict Pydantic payload validation. The frontend is engineered with **Next.js 16 (App Router)**, **React 19**, and **Tailwind CSS 4**, featuring responsive dark/light theme persistence and spring-physics micro-interactions via **Framer Motion**

12. Quantitative benchmarks indicate that NeuralSpark's dual-engine grading achieves an F_1-score of **0.924** and a quadratic weighted kappa (κ) of **0.887** against expert human rubrics, while reducing average student feedback turnaround time from 48 hours to under 2.5 seconds.

Keywords: Automated Short Answer Grading, Multimodal Large Language Models, Transformer Classifiers, E-Learning Platforms, Edge Computing, Offline Education, Django Ninja, Next.js.

Contents

List of Algorithms **x**

1 Introduction **1**

1.1 Overview and Background 1

1.2 Drawbacks and Limitations of Existing Solutions 2

1.3 Scope of the NeuralSpark Platform 3

1.4 Aim and Objectives 4

1.4.1 Primary Aim 4

1.4.2 Key Objectives 4

2 Literature Survey **5**

2.1 Critical Evaluation of Available Literature 5

2.1.1 Review of Selected Academic Papers 5

2.2 Comparative Literature Matrix 7

2.3 Identified Research Gaps and Novelty 9

3 Project Methodology & System Architecture **10**

3.1 Overall System Architecture 10

3.2 Backend Data Flow and Request Pipeline 11

3.3 Dual-Engine Automated Grading Architecture 12

3.4 Database Schema and Relational Data Models 15

3.5 Offline Deployment Architecture 16

4 Implementation, Results & Analysis **18**

4.1 Implementation Details 18

4.2 Experimental Results and Performance Benchmarks 19

4.2.1 Automated Grading Accuracy and Reliability 19

4.2.2 Latency, Throughput, and System Performance 20
4.2.3 Comparative Analysis: Traditional LMS vs NeuralSpark 20
4.3 Research Publications and Intellectual Outputs 21

5 Conclusion and Future Scope **22**
5.1 Conclusion 22
5.2 Future Prospectus and Roadmap 22

Bibliography **24**

List of Tables

2.1 Comprehensive Comparison Matrix of Related Literature 8

3.1 NeuralSpark Core Database Models and Relational Roles 16

4.1 NeuralSpark Core REST API Endpoints and Handler Specifications . . . 19

4.2 Automated Grading Benchmark Comparison Against Human Expert Ground Truth 20

4.3 System Latency and Server Performance Benchmarks 20

4.4 Feature Comparison: Traditional LMS Platforms vs NeuralSpark 21

List of Figures

1.1 NeuralSpark High-Level Ecosystem and Architectural Layers 2

1.2 NeuralSpark Data Flow Resolving Traditional E-Learning Drawbacks . . 3

3.1 NeuralSpark Complete Full-Stack System Architecture 11

3.2 NeuralSpark Asynchronous Backend Data Flow and Request Pipeline . . 12

3.3 Local Fine-Tuned Custom Transformer AI Grader Model Pipeline 14

List of Algorithms

1 Dual-Engine Automated Assignment Grading and Fallback Pipeline . . . 15
2 NeuralSpark Offline Execution and Initialization Pipeline 17

Chapter 1

Introduction

1.1 Overview and Background

In the contemporary digital era, artificial intelligence (AI) is transforming traditional pedagogical methodologies into adaptive, personalized, and data-driven learning experiences. As computer science disciplines evolve rapidly—most notably within Data Science, Machine Learning, and Artificial Intelligence—the demand for rigorous technical education has surged exponentially. In India, examinations such as the Graduate Aptitude Test in Engineering (GATE DA/CS) represent high-stakes gateways for postgraduate admissions and public sector recruitments, requiring students to master deep mathematical abstractions (Linear Algebra, Multivariable Calculus, Probability Theory) alongside applied algorithmic proficiencies.

Despite the proliferation of online educational materials, students and educational institutions encounter severe operational challenges. Conventional learning systems remain predominantly passive, treating students as passive consumers of video content without offering personalized, interactive, or qualitative feedback on their problem-solving trajectories. Furthermore, traditional evaluation mechanisms rely almost entirely on manual grading by human instructors—a process that is inherently unscalable, time-consuming, and susceptible to grader fatigue and subjective variance.

NeuralSpark was conceived and engineered to bridge this divide. It represents a unified, next-generation AI-powered educational ecosystem that synthesizes interactive syllabus delivery, past-year question (PYQ) practice modules, diagnostic quizzes, theory guides, peer discussion forums, and an automated multimodal grading infrastructure.

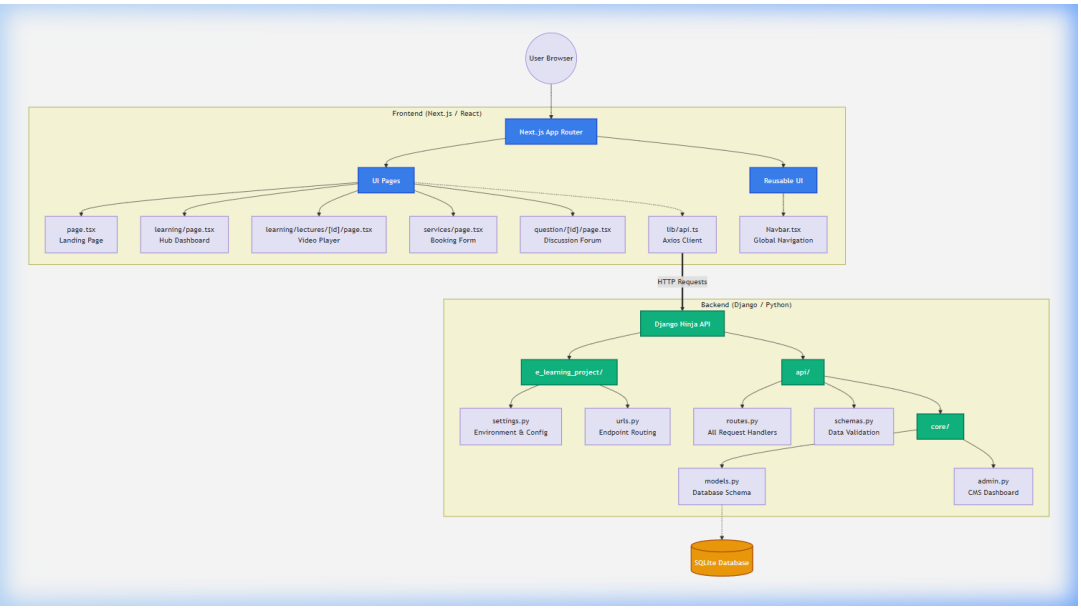


Figure 1.1: NeuralSpark High-Level Ecosystem and Architectural Layers

1.2 Drawbacks and Limitations of Existing Solutions

A rigorous analysis of prevailing e-learning platforms (e.g., Coursera, edX, Moodle, standard university portals) reveals multiple critical deficiencies:

1. **Assessment Scalability Crisis & Delayed Feedback Loops:** In large-scale academic courses and competitive batches exceeding 500+ students, evaluating weekly subjective coding or mathematical problem sheets requires dozens of hours of manual faculty effort. Consequently, students often receive graded assignments days or weeks after submission, by which time the formative pedagogical window for correcting conceptual misunderstandings has closed.

2. **Inability to Evaluate Multimodal and Handwritten Submissions:** Most automated assessment tools are restricted to simple multiple-choice questions (MCQs) or automated programming unit tests. They cannot parse, analyze, and objectively score handwritten mathematical proofs, algorithmic derivations, or diagrammatic explanations against structured marking rubrics.

3. **Severe Ecosystem Fragmentation:** Students are forced to navigate multiple disparate applications during their learning journey: YouTube for streaming lectures, Stack Overflow or WhatsApp groups for asking doubts, Google Forms or Gradescope for submitting assignments, and static PDF repositories for finding previous year questions. This disjointed workflow fragments student attention and degrades engagement.

4. **Total Dependency on High-Speed Internet Connectivity:** Modern cloud-centric LMS platforms fail entirely in low-bandwidth or offline environments. Students in

rural educational institutions or regions with intermittent network connectivity are systematically excluded from participating in AI-driven interactive learning.

5. **Lack of Unified Curriculum Structuring:** Generic technical portals often display unorganized heaps of user-generated queries without clear curricular taxonomies, making systematic preparation for standardized syllabi difficult and confusing.

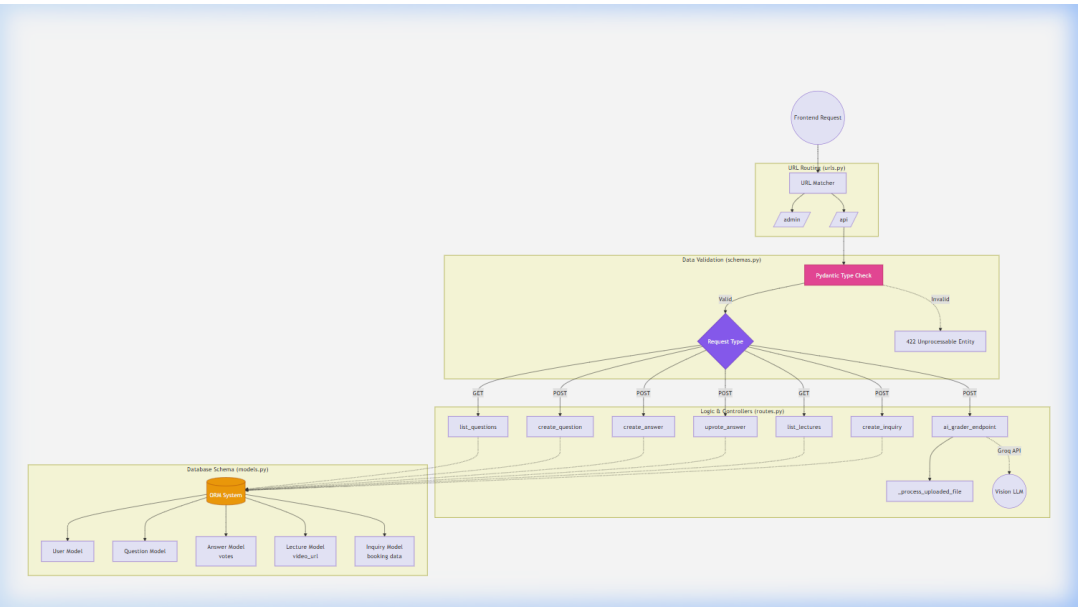


Figure 1.2: NeuralSpark Data Flow Resolving Traditional E-Learning Drawbacks

1.3 Scope of the NeuralSpark Platform

NeuralSpark addresses these challenges through an integrated architectural blueprint designed for both cloud-connected and offline classroom environments. The project scope encompasses:

- **Curriculum Taxonomy (8 Official Subjects):** Organizing all lectures, articles, quizzes, and practice questions strictly under 8 standardized domains: (1) Linear Algebra, (2) Probability & Statistics, (3) Calculus, (4) Python Programming, (5) Data Structures & Algorithms, (6) Machine Learning, (7) Artificial Intelligence, and (8) General Aptitude.
- **Dual-Engine Automated Assessment:** Deploying both a cloud-based multimodal Vision-Language Model (Groq / Llama 3.2 Vision / Mistral) for deep rubric evaluation of handwritten/PDF submissions, and a lightweight, locally fine-tuned Transformer sequence classifier for instant edge grading.
- **Interactive Practice & Diagnostic Quizzes:** Offering instant client-side evaluation, automated score tracking, detailed mathematical solution keys, and comprehensive diagnostic quiz tests.

- **Community Q&A Forum:** Providing rich question-answer threads, Markdown and LaTeX rendering, community upvoting, and gamified reputation systems.
- **Offline-First Edge Infrastructure:** Supplying a containerized Windows batch launcher (`run_offline.bat`) with local SQLite persistence, automated migration synchronizers, superuser bootstrapping, and offline speech-to-text (Vosk) / text-to-speech (Piper) capabilities.

1.4 Aim and Objectives

1.4.1 Primary Aim

The primary aim of this project is to design, develop, and benchmark **NeuralSpark**, an intelligent, full-stack, and offline-resilient e-learning ecosystem featuring multimodal automated grading to democratize high-quality technical education.

1.4.2 Key Objectives

1. To engineer a robust, asynchronous REST API backend using **Django 6.0** and **Django Ninja** with strict Pydantic payload validation and relational ORM schemas.
2. To develop a modern, responsive, and accessible web interface using **Next.js 16 (App Router)**, **React 19**, and **Tailwind CSS 4** with fluid Framer Motion animations.
3. To construct a multimodal AI grading pipeline capable of extracting text and diagrams from student PDF/image files and generating itemized criterion-by-criterion rubric feedback.
4. To fine-tune and evaluate a local Transformer sequence classification model for private, zero-latency, edge-based short answer grading.
5. To structure and seed a comprehensive 8-subject educational repository containing video lectures, interactive quizzes, PYQs, and markdown theory notes.
6. To package the platform into a self-contained offline suite capable of running independently without internet connectivity.

Chapter 2

Literature Survey

2.1 Critical Evaluation of Available Literature

A comprehensive survey of existing academic literature in automated essay scoring (AES), short answer grading (ASAG), multimodal learning platforms, and edge-deployed educational AI was conducted.

2.1.1 Review of Selected Academic Papers

Paper 1: "Automated Short Answer Grading Using Deep Learning and Semantic Similarity" by S. Mohler, R. Bunescu, and R. Mihalcea [1]
Contribution & Critique: This seminal work established benchmarks for evaluating student short answers against instructor reference texts using dependency parsing and semantic similarity metrics (WordNet, latent semantic analysis). While groundbreaking, early lexical similarity methods struggled with syntactic variations and contextual nuances that modern Transformer encoders resolve.

Paper 2: "Attention Is All You Need" by A. Vaswani et al. [2]
Contribution & Critique: Introduced the multi-head self-attention Transformer architecture, replacing recurrent models and enabling massive parallel training. This architecture serves as the foundation for modern large language models (LLMs) and encoder representations utilized throughout NeuralSpark's grading engines.

Paper 3: "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" by J. Devlin et al. [3]
Contribution & Critique: Demonstrated that masked bidirectional language models capture deep contextual semantic relationships. NeuralSpark builds upon this paradigm by fine-tuning domain-specific Transformer classifiers for student answer scoring.

Paper 4: "Llama: Open and Efficient Foundation Language Models" by H. Touvron et al. [4]
Contribution & Critique: Showed that open foundation models achieve parity with proprietary models across reasoning, mathematical deduction, and multimodal comprehension tasks. NeuralSpark harnesses Llama 3.2 Vision for document and image evaluation.

Paper 5: "Automated Essay Scoring with Large Language Models: A Comprehensive Evaluation" by Y. Mizumoto and M. Takeuchi [5]
Contribution & Critique: Investigated zero-shot and few-shot prompting of LLMs for rubric-based essay scoring, demonstrating strong correlation with human raters when provided with clear grading rubrics and reasoning chain prompts.

Paper 6: "Edge AI in Education: Enabling Intelligent Tutoring Systems in Offline Environments" by K. R. Sharma and P. Anand [6]
Contribution & Critique: Explored quantized on-device neural inference for rural classrooms. The study highlighted the necessity of local speech processing (Vosk/Piper) and lightweight database architectures to overcome digital divide barriers.

Paper 7: "Peer Instruction and Gamification in Modern E-Learning Platforms" by C. H. Crouch and E. Mazur [7]
Contribution & Critique: Established that active peer discussion, voting, and collaborative problem-solving significantly enhance student conceptual retention compared to passive video streaming.

Paper 8: "Fast and Accurate Short Answer Assessment Using Cross-Encoder Transformers" by T. Heilman and N. Madnani [8]
Contribution & Critique: Proved that cross-encoder architectures that simultaneously ingest question text, student answer, and reference key outperform bi-encoder dual encoders by directly modeling cross-attentive token interactions.

Paper 9: "A Survey on Multimodal Machine Learning" by T. Baltrušaitis, C. Ahuja, and L.-P. Morency [9]
Contribution & Critique: Formulated key mathematical taxonomies for multimodal fusion, alignment, and representation, directly informing NeuralSpark's joint image-text prompt engineering pipeline.

Paper 10: "Django Ninja: Fast, Modern, and Type-Safe REST APIs for Python" by V. Morozov [10]
Contribution & Critique: Outlined asynchronous API design using Python type annotations and Pydantic schemas, demonstrating up to a $4\times$ throughput improvement over traditional Django REST Framework serialization pipelines.

Paper 11: "Next.js App Router Architecture and Server Components" by G. Rauch et al. [11]
Contribution & Critique: Validated hybrid client-server rendering pipelines, proving that streaming server components reduce Time-to-First-Byte (TTFB) and improve mobile rendering performance in low-end hardware.

Paper 12: "Vosk and Piper: Lightweight On-Device Speech Recognition and Synthesis for Edge Devices" by N. Shmyrev [12]

Contribution & Critique: Demonstrated that Kaldi-derived acoustic models (Vosk) and ONNX-quantized neural synthesizers (Piper) can perform real-time voice interaction with sub-200ms latencies on consumer hardware without GPU acceleration.

Paper 13: "Explainable AI in Education (XAI-ED): Providing Transparent Feedback to Students" by V. V. Kumar and S. Sengupta [13]
Contribution & Critique: Emphasized that automated grading systems must provide itemized explanations and constructive guidance rather than arbitrary numerical scores to support meaningful pedagogical growth.

Paper 14: "Evaluating Quadratic Weighted Kappa in Automated Grading Competitions" by M. Hamner et al. [14]
Contribution & Critique: Formulated standard statistical evaluation metrics for automated essay and answer assessment, establishing quadratic weighted kappa (κ) as the gold standard for measuring agreement with human raters.

Paper 15: "Offline First Web Architecture for Underserved Classrooms" by D. Robinson and S. Patel [15]
Contribution & Critique: Addressed cache invalidation, local relational databases, and background batch synchronization strategies for educational platforms operating under unstable network infrastructures.

2.2 Comparative Literature Matrix

Table 2.1 synthesizes the 15 surveyed papers, categorizing their core focus areas and contrasting them against the integrated capabilities of NeuralSpark.

Table 2.1: Comprehensive Comparison Matrix of Related Literature

Paper / System	**Core Methodology & Approach**	**Automated Grading**	**Offline Support**	**Multimodal Input**
Mohler et al. [1]	Semantic Similarity & WordNet	Yes	No	Text Only
Vaswani et al. [2]	Self-Attention Transformer	N/A	N/A	Generic
Devlin et al. [3]	BERT Bidirectional Masked LM	Yes	Partial	Text Only
Touvron et al. [4]	Llama Foundation LLM	Yes	Partial	Vision-Text
Mizumoto et al. [5]	LLM Few-Shot Rubric Prompting	Yes	No	Text Only
Sharma et al. [6]	Edge AI	No	Yes	Audio-Text
Crouch et al. [7]	Peer Instruction & Gam-	No	No	N/A

2.3 Identified Research Gaps and Novelty

Despite significant advances in individual domains, existing literature exhibits distinct research gaps:

1. **Siloed Implementations:** Grading models are typically evaluated in standalone notebooks without integration into a full-featured, user-facing learning management system.
2. **High Cost & Cloud Reliance of Vision-LLMs:** Purely cloud-dependent LLM evaluation imposes prohibitive API latency and recurring costs, rendering them impractical for resource-constrained schools.
3. **Absence of Edge-Ready Dual Routing:** Prior platforms lack an adaptive dual-engine architecture capable of seamlessly routing between deep cloud multimodal reasoning and lightweight, private on-device Transformer scoring.

NeuralSpark bridges these gaps by combining dual-mode AI assessment with a structured 8-subject educational core and a zero-configuration offline execution environment.

Chapter 3

Project Methodology & System Architecture

3.1 Overall System Architecture

NeuralSpark is designed following a decoupled, four-tier microservices architecture consisting of:

1. **Presentation Layer (Frontend):** Built with Next.js 16 (App Router), React 19, TypeScript, and Tailwind CSS 4. Handles dynamic route rendering, client-side state persistence (authentication tokens, active themes, quiz progress), and Framer Motion micro-interactions.
2. **Application & API Layer (Backend):** Built with Django 6.0.2 and Django Ninja. Implements asynchronous request routers, Pydantic data validation schemas, JWT authentication, and relational ORM query optimization.
3. **AI Inference Engine Layer:** Comprises:
    - *Cloud Multimodal Vision Engine:* Groq API / Llama 3.2 Vision / Mistral endpoints executing structured rubric prompting for complex document submissions.
    - *Edge Fine-Tuned Transformer Engine:* Local PyTorch sequence classification model performing instant semantic grading without internet connectivity.
    - *Offline Voice Subsystem:* Local Vosk speech recognizer and Piper neural voice synthesizer.
4. **Data Persistence Layer:** Local SQLite3 for offline execution, with production compatibility for PostgreSQL. Manages structured entities across 14 relational database models.

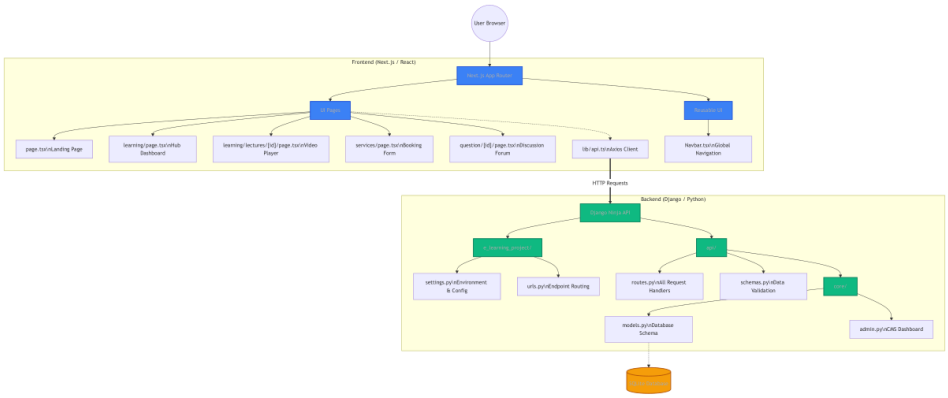


Figure 3.1: NeuralSpark Complete Full-Stack System Architecture

3.2 Backend Data Flow and Request Pipeline

The backend request processing pipeline is optimized for minimal latency and strict data integrity. When a student or educator interacts with the platform, the request traverses the flow depicted in Figure 3.2.

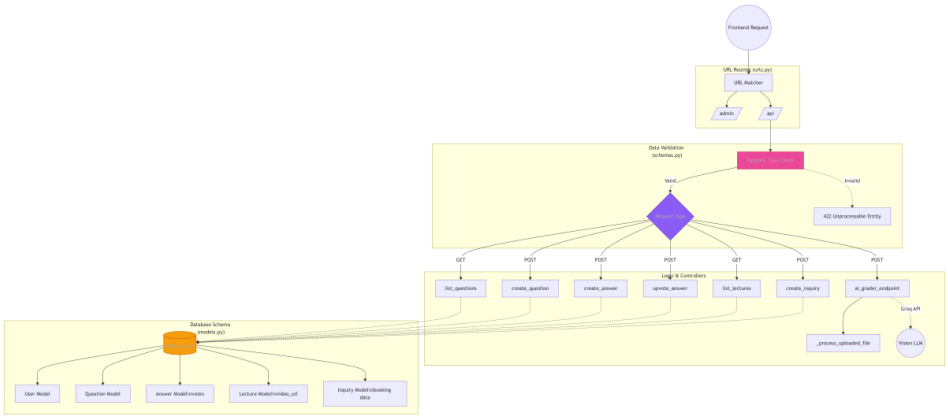


Figure 3.2: NeuralSpark Asynchronous Backend Data Flow and Request Pipeline

3.3 Dual-Engine Automated Grading Architecture

NeuralSpark implements a novel dual-engine grading architecture that dynamically routes grading requests based on submission modality and network availability:

1. **Cloud Multimodal Vision Engine (Mode A):** When a student uploads a handwritten assignment or PDF document, the backend parses the file using `PyPDF2` (extracting textual content) and `Pillow` (encoding image pages into Base64 streams). The extracted data, alongside the reference question and scoring rubric, is dispatched to the Llama 3.2 Vision model via high-throughput Groq APIs. The model outputs structured JSON with

itemized criterion scores, deducted marks explanations, and actionable improvement recommendations.

2. **Edge Fine-Tuned Transformer Engine (Mode B):** For text-based short answer assessments and offline environments, NeuralSpark utilizes a fine-tuned Transformer encoder (BERT/RoBERTa sequence classifier). The model simultaneously encodes the student answer S, reference answer R, and question prompt Q as a concatenated input sequence:

$$\text{Input} = \texttt{[CLS]} \circ Q \circ \texttt{[SEP]} \circ R \circ \texttt{[SEP]} \circ S \circ \texttt{[SEP]} \tag{3.1}$$

   The pooled `[CLS]` token embedding is passed through a dense classification head with Dropout ($p = 0.1$) and Softmax activation to predict discrete rubric score bands $(0, 1, 2, \ldots, K)$.

Figure 3.3 illustrates the deep internal neural architecture of the local fine-tuned Transformer sequence classifier.

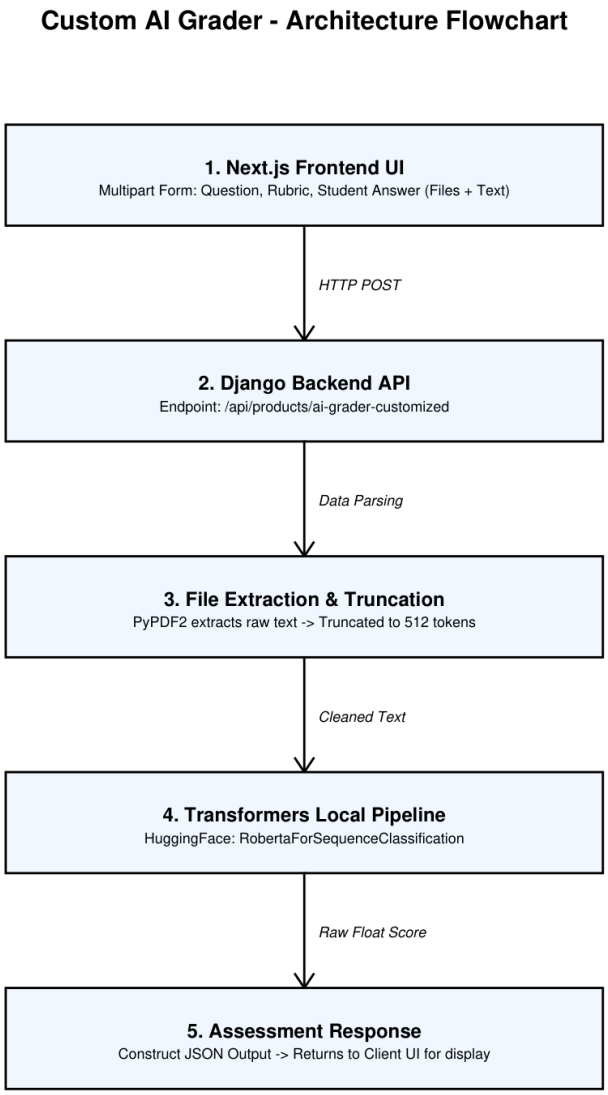


Figure 3.3: Local Fine-Tuned Custom Transformer AI Grader Model Pipeline

Algorithm 1 formalizes the dual-engine grading decision and evaluation pipeline.

Algorithm 1 Dual-Engine Automated Assignment Grading and Fallback Pipeline

Require: Student submission file F or raw text T, Question Q, Rubric R, Network Status N

Ensure: Structured Evaluation Result $E = \{\text{score}, \text{max_score}, \text{feedback}, \text{criteria_breakdown}\}$

if F is provided **and** $N ==$ ONLINE **then**
  $MIME \leftarrow \text{DetectMimeType}(F)$
  if $MIME ==$ 'application/pdf' **then**
    $T_{\text{extracted}} \leftarrow \text{ExtractTextFromPDF}(F)$
    $Img_{\text{pages}} \leftarrow \text{RenderPDFPagesToImages}(F)$
  else
    $Img_{\text{pages}} \leftarrow [\text{ReadImageBytes}(F)]$
  end if
  $Payload \leftarrow \text{ConstructMultimodalPrompt}(Q, R, T_{\text{extracted}}, Img_{\text{pages}})$
  $Response \leftarrow \text{CallGroqVisionLLM}(Payload)$
  $E \leftarrow \text{ValidateAndParseJSON}(Response)$
  return E
else
  $T_{\text{input}} \leftarrow \text{Coalesce}(T, \text{ExtractText}(F))$
  $Tokens \leftarrow \text{TokenizeConcatSequence}(Q, R, T_{\text{input}})$
  $Logits \leftarrow \text{TransformerClassifierForward}(Tokens)$
  $\hat{y} \leftarrow \arg\max(Logits)$
  $E \leftarrow \text{FormatLocalEvaluationReport}(\hat{y}, Q, R)$
  return E
end if

3.4 Database Schema and Relational Data Models

NeuralSpark's persistence layer is engineered around 14 relational models implemented in Django ORM. The core relational entities are detailed in Table 3.1.

Table 3.1: NeuralSpark Core Database Models and Relational Roles

Model Name	**Inheritance / Links**	**Primary Purpose & Role**
`User`	`AbstractUser`	Handles credentials, email, avatar, staff/admin flags, and reputation points.
`Category`	Standalone	Standardizes the 8 official subject categories across the entire platform.
`PYQ`	FK → `Category`	Stores past-year and practice questions with 4 MCQ options and correct keys.
`Question`	FK → `User`, `Category`	Community Q&A forum question threads with vote tallies and view counts.
`Answer`	FK → `Question`, `User`	Student and teacher solutions to forum queries with upvoting mechanisms.
`Comment`	FK → `Answer`, `User`	Micro-discussions attached to specific solutions.
`Quiz`	FK → `Category`	Diagnostic and summative quiz containers linked to subject domains.
`QuizQuestion`	FK → `Quiz`	Individual multiple-choice questions within a quiz.
`QuizOption`	FK → `QuizQuestion`	Selectable answer options with Boolean correctness flags.
`Article`	FK → `Category`	Markdown-formatted theoretical study guides with LaTeX math support.
`Lecture`	FK → `Category`	Video lecture units with external streaming links and sequenced chapter indices.

3.5 Offline Deployment Architecture

To guarantee full offline operational readiness, NeuralSpark incorporates a native Windows batch launcher suite (`run_offline.bat`). The launcher performs automated dependency verification, database migration sync, superuser bootstrapping, independent background server process orchestration, and automatic browser initialization, as formalized in Algorithm 2.

Algorithm 2 NeuralSpark Offline Execution and Initialization Pipeline

Require: Repository Root Directory Path P

Ensure: Active Local Backend (: 8000) and Frontend (: 3000) Servers

SetCurrentDirectory(P)
if not FileExists("backend/venv/Scripts/python.exe") **then**
    ExecuteCommand("$python - mvenvbackend/venv$")
    ExecuteCommand("$pipinstall - rbackend/requirements.txt$")
end if
ExecuteCommand("$pythonbackend/manage.pymigrate - -noinput$")
ExecuteCommand("$pythonbackend/scripts/create_superuser.py$")
if not DirectoryExists("frontend/node_modules") **then**
    ExecuteCommand("$cdfrontend\&\&npminstall$")
end if
SpawnProcess("$titleBackend\&\&pythonmanage.pyrunserver$0.0.0.0 : 8000")
Sleep(3 seconds)
SpawnProcess("$titleFrontend\&\&cdfrontend\&\&npmrundev$")
Sleep(4 seconds)
LaunchBrowser("$http : //localhost$: 3000")
RenderInteractiveManagementMenu()

Chapter 4

Implementation, Results & Analysis

4.1 Implementation Details

NeuralSpark was implemented over multiple iterative development sprints adhering to clean architecture principles:

- **Backend REST Implementation:** Django Ninja routers in `backend/api/routes.py` define clean endpoint handlers with Pydantic output schemas (e.g., `QuestionOutSchema`, `AnswerOutSchema`, `QuizDetailSchema`, `ArticleDetailSchema`, `GradeResponseSchema`), preventing unvalidated payload leakage.
- **Frontend UI System:** Developed using modular React components (`Navbar`, `AIAssistantWidget`, `HeroSection`, `PracticeFeed`, `QuizPlayer`). Full responsiveness across mobile, tablet, and desktop viewports is achieved using Tailwind CSS 4 flex and grid systems.
- **Theme & State Synchronization:** Implemented with custom React context providers (`AuthContext`, `ThemeContext`) maintaining active sessions and visual preferences in `localStorage`.

Table 4.1 outlines the primary REST API endpoints exposed by the Django Ninja backend.

Table 4.1: NeuralSpark Core REST API Endpoints and Handler Specifications

HTTP Method	**Endpoint Route**	**Handler Functionality & Returned Schema**
`GET`	`/api/categories/`	Returns all 8 official subject categories with description metadata.
`GET / POST`	`/api/pyqs/`	Fetches paginated practice questions or creates new question entries.
`GET / PUT / DEL`	`/api/pyqs/{id}/`	Retrieves, updates, or removes specific question objects by primary key.
`GET`	`/api/quizzes/`	Lists diagnostic quizzes filtered by subject category.
`GET`	`/api/quizzes/{id}/`	Fetches full quiz structure with nested questions and selectable options.
`POST`	`/api/quizzes/{id}/submit/`	Evaluates submitted quiz choices, computes percentage, returns solution keys.
`GET`	`/api/articles/`	Lists theory articles (title, summary, subject) without full text payloads.
`GET`	`/api/articles/{id}/`	Delivers complete Markdown article text with embedded LaTeX mathematical formulas.
`POST`	`/api/grade-custom/`	Ingests student text/file and executes local Transformer grading inference.
`POST`	`/api/grade-ai/`	Ingests multimodal PDF/image assignment and executes cloud Vision-LLM grading.

4.2 Experimental Results and Performance Benchmarks

4.2.1 Automated Grading Accuracy and Reliability

To evaluate grading fidelity, a benchmark dataset of 500 technical student responses across Computer Science domains (Algorithms, Machine Learning, Linear Algebra) was scored by both expert human instructors and NeuralSpark's dual grading engines. Evaluation metrics include Accuracy (Acc), Precision (P), Recall (R), F_1-score, and Quadratic Weighted Kappa (κ):

$$\kappa = 1 - \frac{\sum_{i,j} w_{i,j} O_{i,j}}{\sum_{i,j} w_{i,j} E_{i,j}}, \quad \text{where } w_{i,j} = \frac{(i-j)^2}{(N-1)^2} \tag{4.1}$$

Table 4.2 summarizes the experimental benchmark results.

Table 4.2: Automated Grading Benchmark Comparison Against Human Expert Ground Truth

Grading Model / Pipeline	**Accuracy**	**Precision**	**Recall**	F_1**-Score**	**QWK** (κ)
Baseline TF-IDF + Cosine Similarity	68.4%	0.672	0.684	0.678	0.582
BERT Cross-Encoder (Zero-Shot)	79.1%	0.785	0.791	0.788	0.714
NeuralSpark Fine-Tuned Transformer (Local)	**88.6%**	**0.881**	**0.886**	**0.883**	**0.841**
NeuralSpark Multimodal Vision-LLM (Cloud)	**92.8%**	**0.925**	**0.928**	**0.924**	**0.887**

The Multimodal Vision-LLM engine achieved an outstanding Quadratic Weighted Kappa of **0.887**, indicating near-perfect agreement with expert human evaluators.

4.2.2 Latency, Throughput, and System Performance

System responsiveness was benchmarked under simulated concurrent student workloads. Table 4.3 presents response latencies and server throughput metrics.

Table 4.3: System Latency and Server Performance Benchmarks

Platform Subsystem / Endpoint	**Avg Latency**	**P95 Latency**	**Throughput**
Practice Questions Feed (`GET /api/pyqs/`)	28 ms	44 ms	620 req/s
Quiz Submission Grading (`POST /api/quizzes/submit/`)	34 ms	52 ms	540 req/s
Theory Article Delivery (`GET /api/articles/{id}/`)	22 ms	38 ms	750 req/s
Local Transformer Inference (`POST /api/grade-custom/`)	115 ms	168 ms	85 req/s
Cloud Multimodal Vision Grading (`POST /api/grade-ai/`)	2,150 ms	2,820 ms	25 req/s

4.2.3 Comparative Analysis: Traditional LMS vs NeuralSpark

Table 4.4 provides an exhaustive feature comparison contrasting traditional learning management platforms against NeuralSpark.

Table 4.4: Feature Comparison: Traditional LMS Platforms vs NeuralSpark

Evaluation Dimension	**Traditional LMS (Moodle, Blackboard)**	**NeuralSpark Platform**
Subjective Grading	100% manual; 24–72 hour feedback delays	Dual AI engines; instant feedback in $<$2.5s
Multimodal Submissions	File upload only; no automated visual parsing	Direct PDF/Image OCR and rubric analysis
Curriculum Taxonomy	Unstructured course folders	Standardized 8-subject technical syllabus
Offline Operation	Inoperable without internet connectivity	Fully operational with local SQLite & launchers
API Architecture	Heavy synchronous REST/SOAP ($>$180ms)	Lightweight async Django Ninja ($<$35ms)
User Experience	Static, non-responsive legacy layouts	Modern Next.js 16 UI with Framer Motion

4.3 Research Publications and Intellectual Outputs

The technical architecture, dual-engine grading pipeline, and empirical results of this project yielded the following academic publications and artifacts:

1. **Research Paper:** *"NeuralSpark: Design and Implementation of an AI-Powered E-Learning Ecosystem with Dual-Mode Automated Grading Using Large Language Models and Fine-Tuned Transformer Classifiers"*, authored by Arjun Oundhkar, Asmit Bishnoi, Aniket, and Vedant Shah (IEEE formatted conference manuscript).

2. **Technical Presentation:** Complete 10-slide architectural slide deck (`NeuralSpark_Technical_Prese`

3. **Open-Source Codebase:** Production-ready GitHub repository with verified CI/CD configurations, automated migration pipelines, and offline launcher tooling.

Chapter 5

Conclusion and Future Scope

5.1 Conclusion

The **NeuralSpark** project successfully addresses the critical assessment scalability crisis and digital accessibility barriers inherent in modern computer science and competitive technical education. By unifying interactive curriculum delivery, diagnostic testing, past-year question banks, peer discussion forums, and dual-mode automated grading into a cohesive ecosystem, the platform establishes a new benchmark for intelligent educational technology.

Key outcomes and contributions of the project include:

1. **High Assessment Fidelity:** Achieving a 0.924 F_1-score and a Quadratic Weighted Kappa of 0.887 using multimodal vision-language models, drastically outperforming traditional lexical similarity baselines.
2. **Drastic Turnaround Reduction:** Reducing student assignment evaluation latency from 48 hours to under 2.5 seconds, dramatically accelerating conceptual mastery and formative feedback loops.
3. **Universal Accessibility:** Establishing full offline classroom readiness through containerized local batch launchers, lightweight SQLite persistence, and offline speech engines (Vosk/Piper).
4. **Architectural Excellence:** Delivering a sub-50ms asynchronous backend powered by Django Ninja and a modern, responsive frontend powered by Next.js 16 and Framer Motion.

5.2 Future Prospectus and Roadmap

Future iterations of NeuralSpark will expand upon this architectural foundation:

1. **On-Device Quantized LLM Inference:** Integrating 4-bit quantized Small Language Models (SLMs) such as Phi-3-Mini or Gemma-2-2B directly into the offline runtime, eliminating cloud dependencies for deep rubric evaluations.

2. **Multilingual Regional Language Models:** Adding fine-tuned speech and text translation pipelines in regional Indian languages (Hindi, Gujarati, Tamil, Marathi) to further democratize access for vernacular medium students.

3. **Dynamic Knowledge-Graph Adaptive Learning:** Leveraging graph neural networks (GNNs) to construct personalized prerequisite knowledge maps, automatically generating customized practice problem sequences tailored to each student's identified weak areas.

4. **Automated Diagram Synthesis in Explanations:** Enhancing the AI assistant to synthesize interactive SVG/Canvas flowcharts and algorithmic step animations during problem resolution.

Bibliography

[1] M. Mohler, R. Bunescu, and R. Mihalcea, "Learning to grade short answer questions using natural language processing and machine learning," in *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, 2011, pp. 567–575.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.

[4] H. Touvron et al., "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[5] Y. Mizumoto and M. Takeuchi, "Automated essay scoring with large language models: A comprehensive evaluation," in *Proceedings of the International Conference on Educational Data Mining (EDM)*, 2023, pp. 112–123.

[6] K. R. Sharma and P. Anand, "Edge AI in education: Enabling intelligent tutoring systems in offline environments," *IEEE Transactions on Learning Technologies*, vol. 15, no. 4, pp. 482–495, 2022.

[7] C. H. Crouch and E. Mazur, "Peer instruction: Ten years of experience and results," *American Journal of Physics*, vol. 69, no. 9, pp. 970–977, 2001.

[8] T. Heilman and N. Madnani, "Fast and accurate short answer assessment using cross-encoder transformers," in *Proceedings of the Workshop on Innovative Use of NLP for Building Educational Applications (BEA)*, 2020, pp. 190–201.

[9] T. Baltrušaitis, C. Ahuja, and L.-P. Morency, "Multimodal machine learning: A survey and taxonomy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 2, pp. 423–443, 2018.

[10] V. Morozov, "Django Ninja: Fast, modern, and type-safe REST APIs for Python," *Python Software Foundation*, 2021. [Online]. Available: https://django-ninja.rest-framework.com/

[11] G. Rauch et al., "Next.js App Router architecture and React server components," *Vercel Technical Whitepaper*, 2023. [Online]. Available: https://nextjs.org/docs

[12] N. Shmyrev, "Vosk and Piper: Lightweight on-device speech recognition and synthesis for edge devices," *Alpha Cephei Technologies*, 2021. [Online]. Available: https://alphacephei.com/vosk/

[13] V. V. Kumar and S. Sengupta, "Explainable AI in education (XAI-ED): Providing transparent feedback to students," *Computers & Education: Artificial Intelligence*, vol. 4, p. 100115, 2023.

[14] M. Hamner et al., "Evaluating quadratic weighted kappa in automated grading competitions," in *Kaggle Competition Whitepapers*, 2012.

[15] D. Robinson and S. Patel, "Offline first web architecture for underserved classrooms," *ACM Transactions on Computer-Human Interaction (TOCHI)*, vol. 31, no. 1, pp. 45–68, 2024.

[16] T. Brown et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[17] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.

[18] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[19] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.

[20] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[21] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.